

Algorithm Properties & Complexity Analysis

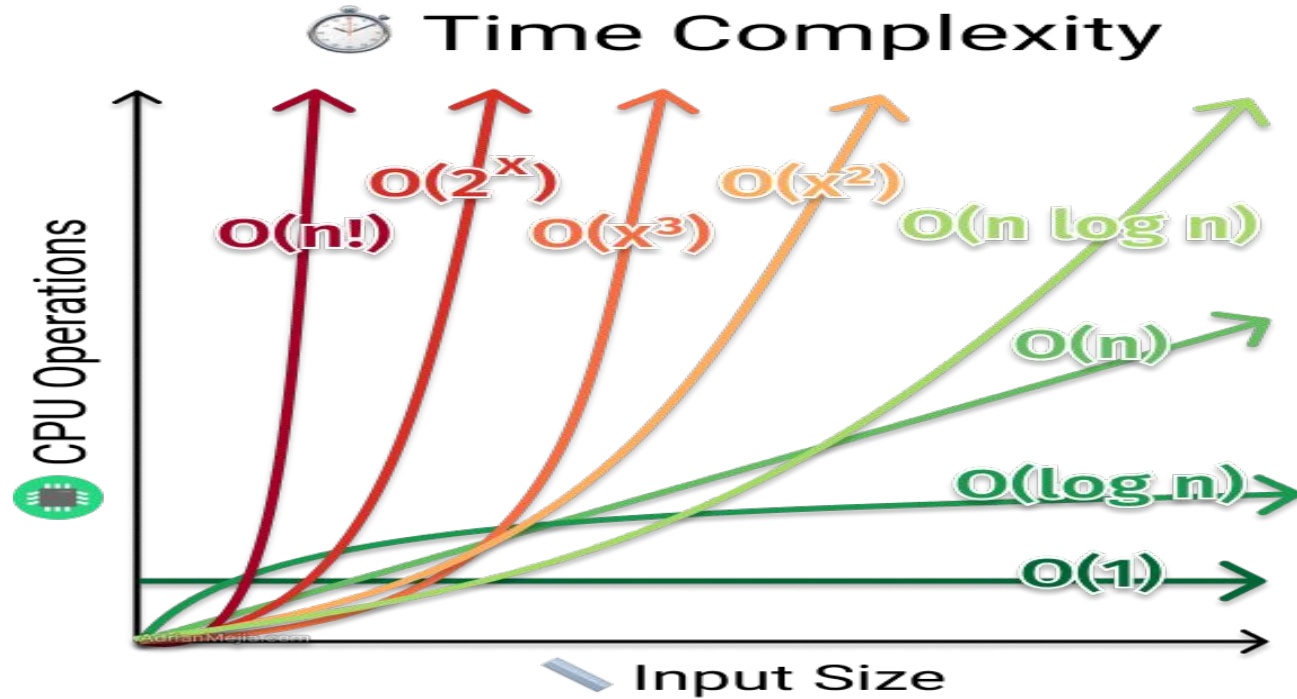
Abdullah All Mamun Siam
Lecturer, DIU

Asymptotic Notations

Some common asymptotic notations –

Constant	$O(1)$
logarithmic	$O(\log n)$
linear	$O(n)$
quadratic	$O(n^2)$
cubic	$O(n^3)$
polynomial	$n^{O(1)}$
exponential	$2^{O(n)}$

Asymptotic Notations



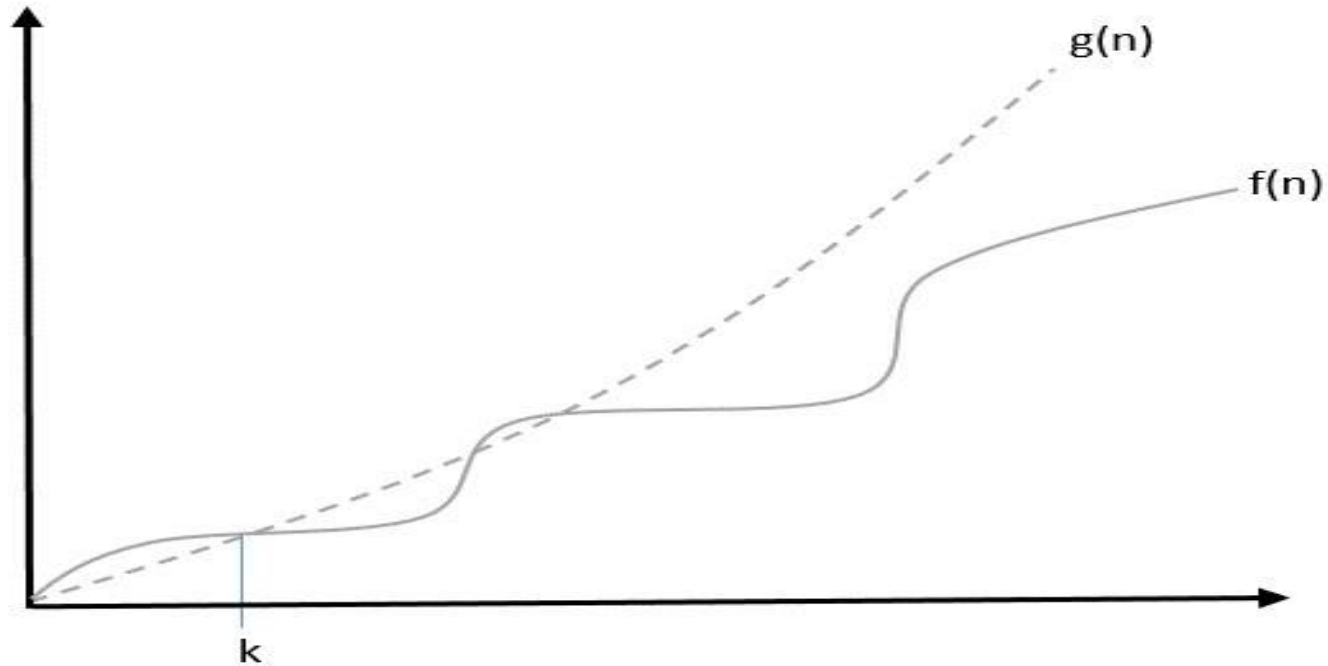
Big Oh, O: Asymptotic Upper Bound

A function **f(n)** can be represented is the order of **g(n)** that is **O(g(n))**, if there exists a value of positive integer **n** as **n₀** and a positive constant **c** such that –

$$f(n) \leq c \cdot g(n) \text{ for } n > n_0 \text{ in all case}$$

Hence, function **g(n)** is an upper bound for function **f(n)**, as **g(n)** grows faster than **f(n)**.

Big Oh, O: Asymptotic Upper Bound



Big Oh, O: Asymptotic Upper Bound

Let us consider a given function,

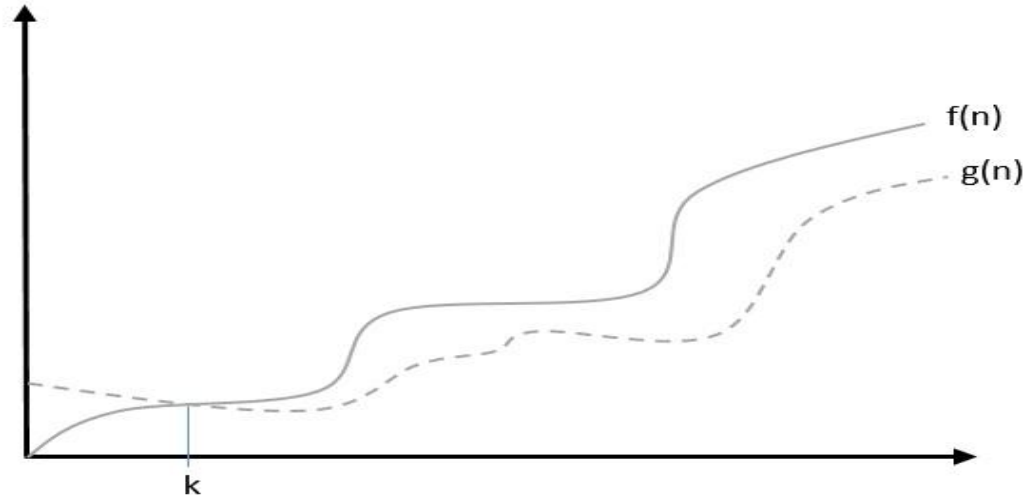
$$f(n)=4n^3+10n^2+5n+1$$

Considering $g(n)=n^3$, $f(n)\leq 5.g(n)$ for all the values of $n>2$

Hence, the complexity of **f(n)** can be represented as $O(g(n))$, i.e. $O(n^3)$

Big Omega, Ω : Asymptotic Lower Bound

We say that $f(n) = \Omega(g(n))$ when there exists constant c that $f(n) \geq c \cdot g(n)$ for all sufficiently large value of n . Here n is a positive integer. It means function g is a lower bound for function f ; after a certain value of n , f will never go below g .



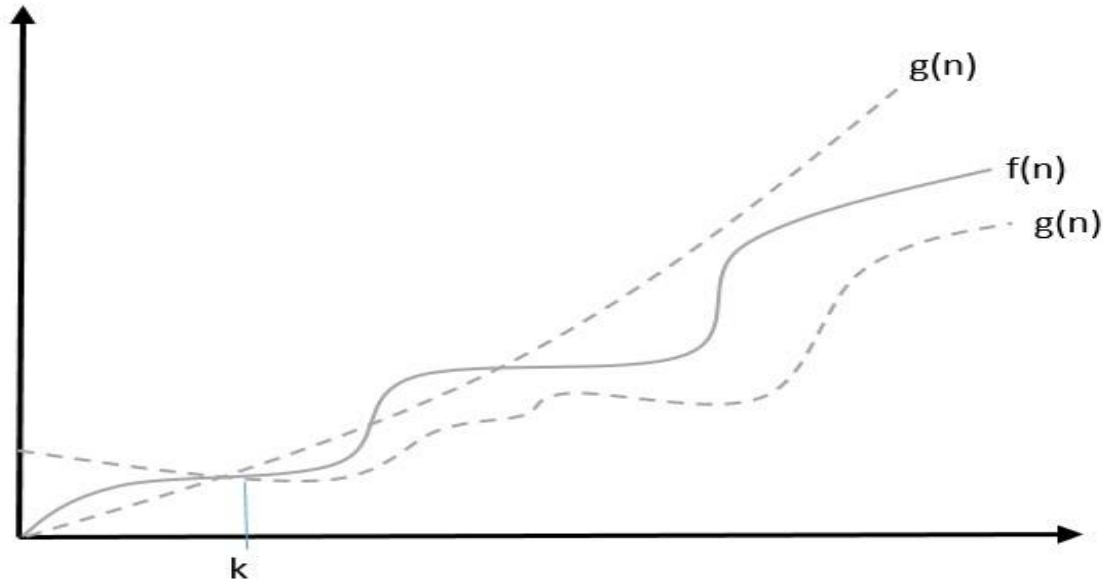
Big Omega, Ω : Asymptotic Lower Bound

Let us consider a given function, $f(n)=4n^3+10n^2+5n+1$

Considering $g(n)=n^3$, $f(n) \geq 4 \cdot g(n)$ for all the values of $n > 0$

Theta, θ : Asymptotic Tight Bound

We say that $f(n)=\theta(g(n))$ when there exist constants c_1 and c_2 that $c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n)$ for all sufficiently large value of n . Here n is a positive integer.



Theta, θ : Asymptotic Tight Bound

Let us consider a given function, $f(n)=4n^3+10n^2+5n+1$

Considering $g(n)=n^3$, $4.g(n)\leq f(n)\leq 5.g(n)$ for all the large values of **n**.

Hence, the complexity of **f(n)** can be represented as $\theta(g(n))$, i.e. $\theta(n^3)$

Find Complexity of An Algorithm

```
int a=0,b=0;
```

```
for(i=0;i<n;i++)
```

```
{
```

```
  a=a+i;
```

```
}
```

$O(n)$ time, $O(1)$ space

Find Complexity of An Algorithm

```
int a = 0, b = 0;  
  
for (i = 0; i < n; i++) {  
    a = a + i;  
}  
  
for (j = 0; j < m; j++) {  
    b = b + j;  
}
```

$O(n + m)$ time, $O(1)$ space

Find Complexity of An Algorithm

```
int a = 0;  
  
for (i = 0; i < n; i++) {  
    for (j = 0; j < n; j++) {  
        a = a + i + j;  
    }  
}
```

$O(n*n)$ time

Find Complexity of An Algorithm

```
int i, j, k = 0;
```

```
for (i = n / 2; i <= n; i++) {
```

```
    for (j = 2; j <= n; j = j * 2) {
```

```
        k = k + n / 2;
```

```
    }
```

```
}
```

$O(n \log n)$ space